Person C – Step-by-step guide (Application: forms, login, validation)

MediSentinel · Phase 1 · Your folder: **app/** · Lead: Rishi (Person A) reviews every pull request

Part 0 – One-time setup (about 45 minutes)

0.1 Install these first

Software	Where
Git	git-scm.com (keep the default options)
Python 3.13	python.org – tick "Add python.exe to PATH"
PostgreSQL 18	postgresql.org/download/windows – remember the **postgres** password you choose
MongoDB 8 Community + Compass	mongodb.com/try/download/community – install as a Windows service
VS Code + Claude Code	code.visualstudio.com, then the Claude Code extension (or the Claude desktop app)

0.2 Accept the invite and clone

Open the GitHub invite e-mail from Rishi and click **Accept**. Then:

Run in the terminal:

```
git clone https://github.com/Rishi799/MediSentinel.git
```

Run in the terminal:

```
cd MediSentinel
```

Open the **MediSentinel** folder in VS Code and start Claude Code in it.

0.3 Let Claude set up your machine

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (app/) and tests/. Never edit sql/, medisentinel/ or
other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.
```

```
- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Task: set up this repo on my Windows machine by following README.md ("Setup" section) step by
step.

1. Create the .venv with Python 3.13, install requirements.txt and run "pip install -e .".

2. Create .env from .env.example. Ask me to choose three passwords for MS_DOCTOR_PASSWORD,

   MS_PHARMACIST_PASSWORD and MS_ADMIN_PASSWORD (do not invent them yourself).

3. Build the database: create the medisentinel database, run sql/schema.sql,
sql/seed_demo.sql and

   sql/roles.sql (with the same three passwords). When the postgres password is needed, ask
me to type it.

4. Run "python tools/check_setup.py" and "python -m pytest".

If anything fails, find the reason and fix it, then run both checks again until everything is
OK.
```

Done when: check_setup prints *All good* and pytest shows all tests passed. If you are stuck, send the error to Rishi.

0.4 Read your task sheet

Paste this prompt into Claude Code:

```
Read docs/tasks/PERSON_C.md, CONTRIBUTING.md and docs/PHASE1_SPRINT.md.

Explain my tasks to me in simple words: what each task produces, in which order I do them,

and which shared functions from medisentinel/ I will use. Do not write any code yet.
```

Task C1 + C2 – Login and validation

Branch: `feature/app-login` · **When:** Day 1 morning

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/app-login
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (app/) and tests/. Never edit sql/, medisentinel/ or
other people's folders.
```

```
- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Tasks C1 and C2 from docs/tasks/PERSON_C.md (Streamlit app).

C1 – app/Home.py and app/session.py:

- Login form (username + password) calling medisentinel.db.connect(username, password).

  On psycopg.OperationalError show "Invalid username or password" (never the raw error).

- On success store the connection and db.role_of(conn) in st.session_state and show "Logged
in as <role>".

- Logout button that closes the connection and clears the session.

- session.py: require_login() stops a page with a message when not logged in;
require_role(*roles).

C2 – app/validation.py with pure functions returning a list of error messages (empty =
valid):

validate_name (required, letters/spaces/./' only, 2-100 chars), validate_dob (not in future,
age 0-120),

validate_dose (number + unit from medisentinel.constants.DOSE_UNITS, e.g. "500 mg", "2.5
ml"),

validate_items (at least one medicine, no medicine twice, each dose valid, frequency in
FREQUENCIES).

Add tests/test_validation.py with a valid and an invalid example for every rule.

Run python -m pytest and start the app with: streamlit run app/Home.py
```

Step 3 – check it yourself

Run `streamlit run app/Home.py`. Log in with **ms_doctor** and your password from .env – it should say *Logged in as doctor*. A wrong password must show *Invalid username or password*. Logout must work.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch

feature/app-login to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain app/Home.py, session.py and validation.py for the viva: how login works with
PostgreSQL roles (the

database decides what each role can do), what st.session_state is, and why validation is done
in the form AND by

database constraints. Then ask me 3 viva questions.
```

Task C3 – Patients page

Branch: `feature/patient-form` · **When:** Day 1 afternoon

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/patient-form
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (app/) and tests/. Never edit sql/, medisentinel/ or
other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Task C3 from docs/tasks/PERSON_C.md: app/pages/1_Patients.py.

- Start with require_login() from app/session.py.

- st.form with: full name, date of birth (st.date_input), gender (GENDERS), kidney and liver
function

  (ORGAN_FUNCTION), conditions (multiselect from db.list_conditions(conn)), allergies
(multiselect of medicines

  from db.list_medications(conn) plus a reaction text for each).

- On submit: run the validation functions, show ALL errors with st.error and save nothing; if
valid call

  db.add_patient(...) and show st.success("Patient #<id> saved").

- Pharmacists must not see the form (show a read-only note instead). If a write still fails
with

  psycopg.errors.InsufficientPrivilege show "Your role is not allowed to do this".
```

```
- Below the form show a table of db.list_patients(conn).
```

Step 3 – check it yourself

As **ms_doctor**: add a patient with condition *peptic ulcer* – it must appear in the table. Try a future date of birth and an empty name – errors, nothing saved. As **ms_pharmacist**: no form visible.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch

feature/patient-form to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain 1_Patients.py for the viva: how one patient with conditions and allergies is saved as
ONE transaction

(look at db.add_patient), what happens when validation fails, and how the pharmacist is
blocked both in the app

and by the database. Then ask me 3 viva questions.
```

Task C4 – Prescriptions page with safety alerts

Branch: `feature/prescription-form` · **When:** Day 2 morning

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/prescription-form
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:
```

```
- Only change files inside my own folder (app/) and tests/. Never edit sql/, medisentinel/ or
other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Task C4 from docs/tasks/PERSON_C.md: app/pages/2_Prescriptions.py.

- require_login(); choose patient (db.list_patients) and doctor (db.list_doctors) by name,
using their ids.

- "How many medicines?" (1-6), then one row per medicine: medicine selectbox
(db.list_medications),

  dose text, frequency selectbox (FREQUENCIES).

- Validate with validate_items; on success call db.create_prescription(conn, patient_id,
doctor_id, items).

- Show the returned alerts: MAJOR -> st.error (red), MODERATE -> st.warning, MINOR ->
st.info,

  none -> st.success("No safety issues found").

- Friendly messages for database errors: UniqueViolation -> "This medicine is already in the
prescription",

  InsufficientPrivilege -> "Your role is not allowed to do this".
```

Step 3 – check it yourself

As **ms_doctor**: prescribe **Warfarin + Aspirin** to the patient with peptic ulcer – red MAJOR alerts must appear (interaction and contraindication). Choosing the same medicine twice must show an error.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch

feature/prescription-form to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain 2_Prescriptions.py for the viva: the full path of an alert (form ->
db.create_prescription ->
```

```
INSERT into Prescription_Item -> trigger trg_medication_safety -> Alert_Log -> back to the screen), why the

prescription is one transaction, and what the per-patient lock prevents. Then ask me 3 viva questions.
```

Task C5 – Alerts page

Branch: `feature/alerts-page` · **When:** Day 2 afternoon

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/alerts-page
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo: https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (app/) and tests/. Never edit sql/, medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Task C5 from docs/tasks/PERSON_C.md: app/pages/3_Alerts.py.

- require_login(); optional patient filter; show db.get_active_alerts(conn, patient_id) with st.dataframe.

- Doctors only: choose an alert, type a reason, Override button -> db.override_alert(conn, alert_id, reason)

  -> success message and the table refreshes. Empty reason -> error. Pharmacists only see the table.
```

Step 3 – check it yourself

Override one of the Warfarin/Aspirin alerts with a reason – it must disappear from the table. An empty reason must show an error. As **ms_pharmacist** there is no override button.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch

feature/alerts-page to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain 3_Alerts.py for the viva: why a doctor can only mark an alert as overridden but can
never create, edit

or delete one (SECURITY DEFINER trigger + column-level GRANT in sql/roles.sql), and why the
override reason is

kept (audit trail). Then ask me 3 viva questions.
```

Task C6 – Week 7 document: forms and validation

The lab schedule (Week 7) needs a written *description of the forms and their validations*. Rishi writes the security part; you write the forms part. Take screenshots of every page (Windows + Shift + S) – including one screenshot of the validation errors – and save them in `docs/forms/`.

Run in the terminal:

```
git checkout main

git pull

git checkout -b docs/forms-description
```

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (app/ and docs/forms/) and tests/. Never edit sql/,
medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Write docs/forms/FORMS_AND_VALIDATION.md describing every form in app/ for my lab submission:

for each page (Login, Patients, Prescriptions, Alerts) give its purpose, every field with its
input type,
```

```
the validation rule and the exact error message, which roles can use it, and which database function it calls.

Insert my screenshots from docs/forms/ under each page. End with a table of all validation rules.

Then commit (no Co-Authored-By line) and push the branch.
```

Fixing things Rishi asks for in review

Paste this prompt into Claude Code:

```
Rishi left review comments on my pull request. Here they are:

<paste the comments here>

Fix each one in my branch, run python -m pytest, commit (no Co-Authored-By line) and push to the same branch.
```

If something goes wrong

Problem	What to do
password authentication failed	Your .env passwords don't match the ones used in roles.sql – ask Claude to re-run sql/roles.sql with your .env passwords.
MongoDB connection refused	Start the service: Windows search → Services → MongoDB → Start.
ModuleNotFoundError: medisentinel	Activate .venv and run `pip install -e .`
Merge conflict / push rejected	Paste the error into Claude: "Explain this git error and fix it safely without losing my work." If unsure, ask Rishi.

Your checklist

	Done when
☐	Setup: check_setup says All good, tests pass
☐	C1/C2: login, logout, validation functions with tests – PR merged
☐	C3: patient form saves valid patients and rejects invalid input – PR merged
☐	C4: Warfarin + Aspirin shows red MAJOR alerts – PR merged
☐	C5: override with reason works, pharmacist is read-only – PR merged

☐	C6: forms & validation document with screenshots – PR merged
☐	You can explain every page in your own words